

CSE208: Problem on Intractability

Section C

July 12, 2025

Problem Statement

Consider the following decision problem:

Given a boolean formula in conjunctive normal form (CNF) with three literals per clause, determine whether there exists an assignment of truth values to the variables such that each clause has at least one true literal and at least one false literal.

A boolean formula is in **conjunctive normal form (CNF)** if it is a conjunction (heavy name for Boolean AND) of one or more clauses, where each clause is a disjunction (heavy name for Boolean OR) of one or more literals. A **literal** is either a variable or its negation (heavy name for Boolean NOT). A **clause** is a disjunction of literals. For example, the formula $(x_1 \vee \neg x_2 \vee x_5) \wedge (x_2 \vee x_3 \vee \neg x_4)$ is in CNF, where $(x_1 \vee \neg x_2 \vee x_5)$ and $(x_2 \vee x_3 \vee \neg x_4)$ are clauses.

The problem we are considering is known as **NAE-3-SAT** (Not All Equal 3-SAT). The goal is to determine if there exists an assignment of truth values to the variables such that each clause has at least one true literal and at least one false literal. For example, for the formula $(x_1 \vee \neg x_2 \vee x_5) \wedge (x_2 \vee x_3 \vee \neg x_4)$, an assignment such as $x_1 = \text{true}, x_2 = \text{false}, x_3 = \text{true}, x_4 = \text{false}, x_5 = \text{true}$ satisfies the NAE-3-SAT condition.

However, for the formula $(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_2 \vee \neg x_3)$, there is no assignment of truth values that satisfies the NAE-3-SAT condition, as any assignment will lead to at least one clause having all true or all false literals.

Task 1: Construct a boolean formula in CNF with three literals per clause that is not satisfiable under the NAE-3-SAT condition. Provide a brief explanation of why no assignment of truth values can satisfy the condition for your formula.

Proof of NP-Completeness

A decision problem is **NP-complete** if it is in **NP** (nondeterministic polynomial time) and every problem in **NP** can be reduced to it in polynomial time. Alternatively, a problem is **NP-complete**, if it is both in **NP** and **NP-hard**. (Think at home: why?)

First, we will prove that NAE-3-SAT is in NP. A problem is in NP if a given yes-solution can be verified in polynomial time.

Task 2: Prove that NAE-3-SAT is in NP.

Next, we will show that NAE-3-SAT is NP-hard by reducing a known NP-complete problem to it. We will use the 3-SAT problem, which is known to be NP-complete. The 3-SAT problem asks whether a given boolean formula in CNF with three literals per clause can be satisfied by some assignment of truth values to the variables.

To run the reduction $3\text{-SAT} \leq_P \text{NAE-3-SAT}$, we will have to efficiently transform an arbitrary instance of 3-SAT into an ‘equivalent’ instance of NAE-3-SAT. By ‘equivalent’, we mean that the original instance of 3-SAT is satisfiable if and only if the transformed instance of NAE-3-SAT is satisfiable.

Task 3: Describe how to transform an instance of 3-SAT into an ‘equivalent’ instance of NAE-3-SAT. You also have to show this transformation runs in polynomial time. (Hint: you might do a lot of stuff, like splitting every clause into two clauses, adding additional variables, etc.)

Task 4: It’s time for the hard part! Prove that the transformation you described in Task 3 is indeed a polynomial-time reduction from 3-SAT to NAE-3-SAT. In other words, show that

1. If the original 3-SAT instance is satisfiable, then the transformed NAE-3-SAT instance is also satisfiable.
2. If the original 3-SAT instance is not satisfiable, then the transformed NAE-3-SAT instance is also not satisfiable.
OR equivalently, If the transformed NAE-3-SAT instance is satisfiable, then the original 3-SAT instance is also satisfiable.

If you have completed all the tasks, you have shown that NAE-3-SAT is NP-complete.